



沈阳建筑大学

上机实验报告

课程名称:

操作系统

实验题目:

进程调度模拟实验

专业班级:

计算机 1902 班

学 号:

1906410229

姓 名:

蔡艺洵

日 期:

2021 年 9 月 29 号

【实验目的】

1. 理解进程的概念，熟悉进程的组成；
2. 用高级语言编写和调试一个进程调度程序，以加深对进程调度算法的理解。

【实验准备】

1. 五种进程调度算法
 - 短进程优先调度算法
 - 高优先权优先调度算法
 - 先来先服务调度算法
 - 基于时间片的轮转调度算法
2. 进程的组成
 - 进程控制块（PCB）
 - 程序段
 - 数据段
3. 进程的基本状态
 - 就绪 W（Wait）
 - 执行 R（Run）
 - 阻塞 B（Block）

【实验内容】

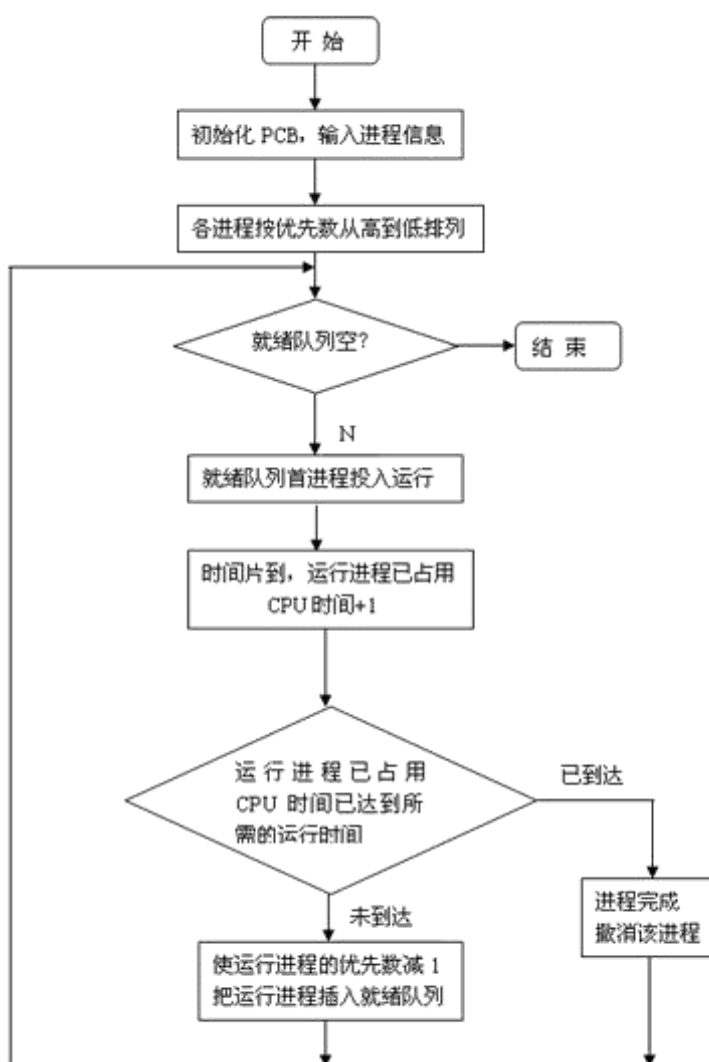
1. 示例

设计一个有 N 个进程共行的进程调度程序。

进程调度算法：采用最高优先数优先的调度算法（即把处理机分配给优先数最高的进程）和先来先服务算法。每个进程有一个进程控制块（PCB）表示。进程控制块可以包含如下信息：进程名、优先数、到达时间、需要运行时间、已用 CPU 时间、进程状态等等。进程的优先数及需要的运行时间可以事先人为地指定（也可以由随机数产生）。进程的到达时间为进程输入的时间。进程的运行时间以时间片为单位进行计算。每个进程的状态可以是就绪 W（Wait）、运行 R（Run）、

或完成 F (Finish) 三种状态之一。就绪进程获得 CPU 后都只能运行一个时间片。用已占用 CPU 时间加 1 来表示。如果运行一个时间片后，进程的已占用 CPU 时间已达到所需要的运行时间，则撤消该进程，如果运行一个时间片后进程的已占用 CPU 时间还未达所需要的运行时间，也就是进程还需要继续运行，此时应将进程的优先数减 1（即降低一级），然后把它插入就绪队列等待 CPU。每进行一次调度程序都打印一次运行进程、就绪队列、以及各个进程的 PCB，以便进行检查。重复以上过程，直到所要进程都完成为止。

2. 调度算法的流程图



【源程序代码】

```
#define _CRT_SECURE_NO_WARNINGS
#include <stdio.h>
#include <stdlib.h>
#include <conio.h>
#define getpch(type) (type*)malloc(sizeof(type))
#define NULL 0
struct pcb { /* 定义进程控制块 PCB */
    char name[10];
    char state;
    int super;
    int ntime;
    int rtime;
    struct pcb* link;
}*ready = NULL, *p;
typedef struct pcb PCB;
void sort() /* 建立对进程进行优先级排列函数*/
{
    PCB *first, *second;
    int insert = 0;
    if ((ready == NULL) || ((p->super) > (ready->super))) /*优先级最大者, 插入队首*/
    {
        p->link = ready;
        ready = p;
    }
    else /* 进程比较优先级, 插入适当的位置中*/
    {
        first = ready;
        second = first->link;
        while (second != NULL)
        {
            if ((p->super) > (second->super)) /*若插入进程比当前进程优先数大,*/
            { /*插入到当前进程前面*/
                p->link = second;
                first->link = p;
                second = NULL;
            }
        }
    }
}
```

```

        insert = 1;
    }
    else /* 插入进程优先数最低, 则插入到队尾*/
    {
        first = first->link;
        second = second->link;
    }
}
if (insert == 0) first->link = p;
}
}

void input() /* 建立进程控制块函数*/
{
    int i, num;
    system("cls");
    //clrscr(); /*清屏*/
    printf("\n 请输入进程号?");
    scanf("%d", &num);
    for (i = 0; i < num; i++)
    {
        printf("\n 进程号 No. %d:\n", i);
        p = getpch(PCB);
        printf("\n 输入进程名:");
        scanf("%s", p->name);
        printf("\n 输入进程优先数:");
        scanf("%d", &p->super);
        printf("\n 输入进程运行时间:");
        scanf("%d", &p->ntime);
        printf("\n");
        p->rtime = 0; p->state = 'w';
        p->link = NULL;
        sort(); /* 调用 sort 函数*/
    }
}

int space()
{
    int l = 0; PCB* pr = ready;
    while (pr != NULL)
    {
        l++;
    }
}

```

```

        pr = pr->link;
    }
    return(l);
}

void disp(PCB * pr) /*建立进程显示函数, 用于显示当前进程*/
{
    printf("\n qname \t state \t super \t ndtime \t runtime \n");
    printf("|%s\t", pr->name);
    printf("|%c\t", pr->state);
    printf("|%d\t", pr->super);
    printf("|%d\t", pr->ntime);
    printf("|%d\t", pr->rtime);
    printf("\n");
}

void check() /* 建立进程查看函数 */
{
    PCB* pr;
    printf("\n **** 当前正在运行的进程是:%s", p->name); /*显示当前运行进程*/
    disp(p);
    pr = ready;
    printf("\n ****当前就绪队列状态为:\n"); /*显示就绪队列状态*/
    while (pr != NULL) {
        disp(pr);
        pr = pr->link;
    }
}

void destroy() /*建立进程撤消函数(进程运行结束, 撤消进程)*/{
    printf("\n 进程 [%s] 已完成. \n", p->name);
    free(p);
}

void running() /* 建立进程就绪函数(进程运行时间到, 置就绪状态)*/{
    (p->rtime)++;
    if (p->rtime == p->ntime)
        destroy(); /* 调用 destroy 函数*/
    else{
        (p->super)--;
        p->state = 'w';
        sort(); /*调用 sort 函数*/
    }
}

void main() /*主函数*/{
    int len, h = 0;
    char ch;
    input();
    len = space();
    while ((len != 0) && (ready != NULL)){

```

```

        ch = getchar();
        h++;
        printf("\n The execute number:%d \n", h);
        p = ready;
        ready = p->link;
        p->link = NULL;
        p->state = 'R';
        check();
        running();
        printf("\n 按任一键继续.....");
        ch = getchar();}
    printf("\n\n 进程已经完成.\n");
    ch = getchar();
}

```

运行截图：

```

C:\Users\蔡某某\source\repos\operatingsystem1\Debug\operatingsystem1.exe
请输入进程号?
5
进程号No. 0:
输入进程名:cyx
输入进程优先数:1
输入进程运行时间:2

进程号No. 1:
输入进程名:hwg
输入进程优先数:2
输入进程运行时间:3

进程号No. 2:
输入进程名:bh1
输入进程优先数:4
输入进程运行时间:4

```

```

C:\Users\蔡某某\source\repos\operatingsystem1\Debug\operatingsystem1.exe
输入进程名:wwj
输入进程优先数:2
输入进程运行时间:6

The execute number:1
**** 当前正在运行的进程是:bh1
qname    state    super    ndtime    runtime
bh1      |R       |4       |4       |0
****当前就绪队列状态为:
qname    state    super    ndtime    runtime
xxy      |w       |3       |3       |0
qname    state    super    ndtime    runtime
hwg      |w       |2       |3       |0
qname    state    super    ndtime    runtime
wwj      |w       |2       |6       |0
qname    state    super    ndtime    runtime
cyx      |w       |1       |2       |0
按任一键继续.....

```

【实验体会】

通过本次的上机实验，让我通过实际去理解关于进程调度算法，去体会不同的调度算法之间的优缺点，永远没有完美的算法，我们只能采用较为优化的算法，来进行进程调度。而且不同的算法之间没有绝对的优劣，而是根据应用场景的不同、应用对象的不同而采用不同的算法。这是在学习进程调度算法的过程中我所感悟的。

【思考题】

几种进程调度算法各有什么优缺点？

1. 时间片轮转调度算法（RR）：给每个进程固定的执行时间，根据进程到达的先后顺序让进程在单位时间片内执行，执行完成后便调度下一个进程执行，时间片轮转调度不考虑进程等待时间和执行时间，属于抢占式调度。优点是兼顾长短作业；缺点是平均等待时间较长，上下文切换较费时。适用于分时系统。
2. 先来先服务调度算法（FCFS）：根据进程到达的先后顺序执行进程，不考虑等待时间和执行时间，会产生饥饿现象。属于非抢占式调度，优点是公平，实现简单；缺点是不利于短作业。
3. 优先级调度算法（HPF）：在进程等待队列中选择优先级最高的来执行。常被用于批处理系统中，还可用于实时系统中。
4. 多级反馈队列调度算法：将时间片轮转与优先级调度相结合，把进程按优先级分成不同的队列，先按优先级调度，优先级相同的，按时间片轮转。优点是兼顾长短作业，有较好的响应时间，可行性强，适用于各种作业环境。
5. 高响应比优先调度算法：根据“响应比=（进程执行时间+进程等待时间）/ 进程执行时间”这个公式得到的响应比来进行调度。高响应比优先算法在等待时间相同的情况下，作业执行的时间越短，响应比越高，满足短任务优先，同时响应比会随着等待时间增加而变大，优先级会提高，能够避免饥饿现象。优点是兼顾长短作业，缺点是计算响应比开销大，适用于批处理系统。